Laboratorio 1: Sistema de Gestión de Territorios Galácticos (STG)

1. Contexto del Proyecto

La startup **"Milky Way Real Estate"** necesita un backend robusto para gestionar la venta de parcelas planetarias dentro de nuestra galaxia. El sistema debe permitir consultar quiénes son los dueños de los planetas y qué porcentaje del suelo poseen.

Reglas de Negocio Críticas:

- **Alcance:** Solo se permiten planetas de la Vía Láctea.
- **Divisibilidad:** Cada planeta representa un 100%.
- **Umbral de Inversión:** Un usuario solo puede aparecer como dueño si posee, como mínimo, el 30% de un planeta. No se permiten micro-transacciones menores a ese porcentaje.

Arquitectura del Backend

Para este laboratorio, se exige una **Arquitectura en Capas** (N-Tier) para asegurar la separación de responsabilidades, y se trabajará exclusivamente en la capa de Controladores:

1. **Capa de Controladores (API/Entry Points):** Manejo de rutas HTTP y validación de entrada.
2. **Capa de Servicio (Business Logic):** Aquí reside la lógica del 30% y la validación de que el planeta pertenezca a la Vía Láctea.
3. **Capa de Acceso a Datos (Repository/Persistence):** Consultas a la base de datos o almacenamiento en memoria.

Fase de Modelado (C4 Model)

Antes de programar, se debe visualizar la estructura, y diseñar los dos primeros niveles del modelo C4.

- **Nivel 1: Diagrama de Contexto**

Muestra cómo el sistema de la API interactúa con los usuarios (Compradores) y sistemas externos (ej. un servicio de verificación de coordenadas galácticas o una base de datos de la NASA).

- **Nivel 2: Diagrama de Contenedores**

Desglosa el sistema. Debe mostrar al menos:

- **Aplicación Web/Mobile:** El frontend que consume la API.
- **API Application (Backend):** El servicio que construirás (Java, Node, Python, etc.).
- **Database:** Donde se guarda la persistencia de usuarios y planetas.

Especificaciones del API

Para que el sistema sea funcional, el API debe poder consultar y se deben implementar validaciones estrictas en el cuerpo de las peticiones (**Request Body**).

Tabla de Endpoints Actualizada

Método	Endpoint	Acción	Regla de Negocio / Validación
POST	/users	Crear Usuario	Debe incluir: nombre, email y documento de identidad.
POST	/planets	Registrar Planeta	**Validación:** El campo galaxy debe ser obligatoriamente "Milky Way".
POST	/ownership	Asignar Propiedad	**Crítico:** No permitir asignaciones de menos de 30%. Validar que el planeta no supere el 100% total.
GET	/planets	Listar Planetas	Devuelve la lista completa de planetas registrados en la Vía Láctea.
GET	/users	Listar Usuarios	Devuelve todos los usuarios, incluyendo sus posesiones actuales.

2. Instrucciones

En su rol de arquitecto, utilice el contexto descrito para la aplicación en la sección 1 para:

1. Crear las primeras 2 capas del diagrama C4 de esta aplicación
2. Documentar el modelo y los métodos del API en Open API

En su rol de desarrollador, utilice los diseños de arquitectura realizados para

3. Implementar la simulación de un API utilizado json-server

Pasos Sugeridos:

1. Instalar Postman https://www.postman.com/downloads/
2. Instalar nodejs https://nodejs.org/en/download/
3. Crear el modelo de un RESTful API, el cual cumpla con los requerimientos señalados en la sección 1

4. Implemente este modelo en un documento de Swagger -> https://editor.swagger.io/
5. Implemente el modelo utilizando json-server para simular un API

Pasos para crear el modelo json-server:

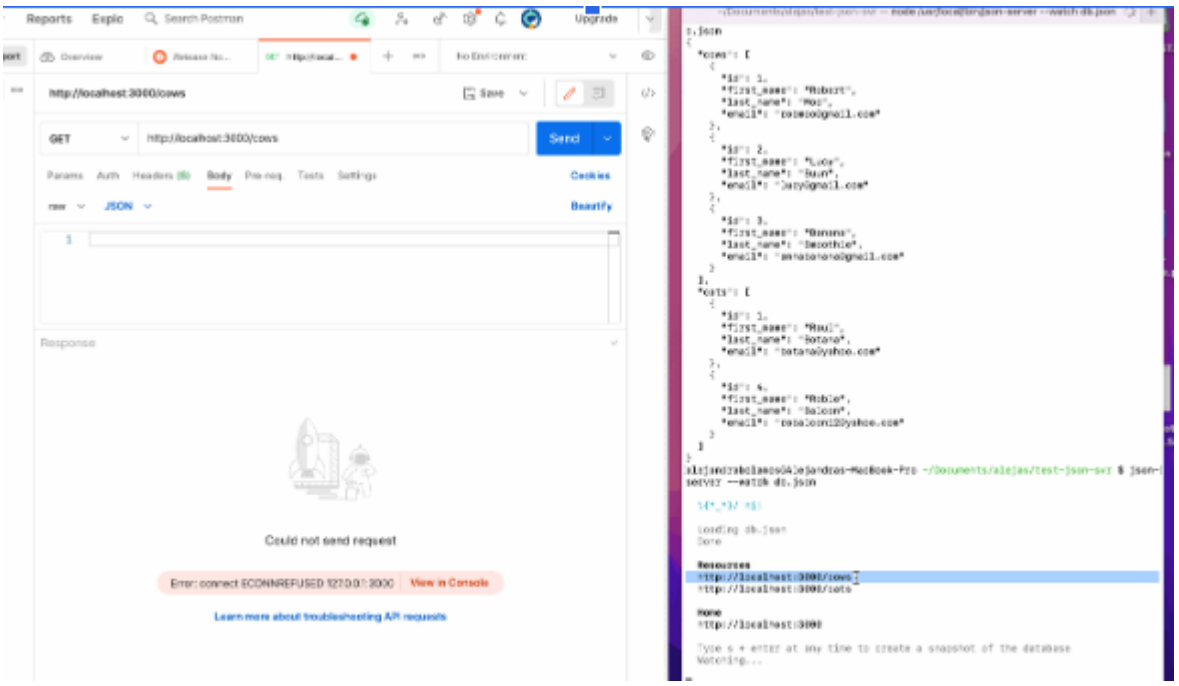
5. Instale json-server usando nodejs :
    npm install -g json-server
6. Ejecute el servidor para que pueda modificar el archivo de configuración creado.
    `json-server --watch db.json`

7. Modifique el db.json y en él cree la simulación de los datos del servicio RESTful que desea simular.
Por ejemplo, mi versión de un método post en este db.json se ve así:

```
[alejandrabolanos@Alejandras-MacBook-Pro ~/Documents/alejas/test-json-svr $ cat d]
b.json
{
  "cows": [
    {
      "id": 1,
      "first_name": "Robert",
      "last_name": "Moo",
      "email": "robmoo@gmail.com"
    },
    {
      "id": 2,
      "first_name": "Lucy",
      "last_name": "Suun",
      "email": "lucy@gmail.com"
    },
    {
      "id": 3,
      "first_name": "Banana",
      "last_name": "Smoothie",
      "email": "annabanana@gmail.com"
    }
  ],
  "cats": [
    {
      "id": 1,
      "first_name": "Raul",
      "last_name": "Botana",
      "email": "botana@yahoo.com"
    },
    {
      "id": 4,
      "first_name": "Roble",
      "last_name": "Balcon",
      "email": "robalcon12@yahoo.com"
    }
  ]
}
```

8. Ejecute el servidor para que pueda consultar la simulación en su máquina local
    `json-server --watch db.json`
9. Consulte la información de la simulación usando los verbos (GET, POST) utilizando Postman

Un ejemplo de respuesta de GET lo puede ver a continuación



Entregables:

1- El documento YAML de Swagger creado en el paso 3
2- El archivo db.json final
3- Un video consultando un método POST, un método GET del servidor simulado. (Puede utilizar LiceCap para grabar el video)

Método de entrega:

- Suba los tres entregables a esta tarea del Tec Digital. Puede ser un archivo comprimido